

File Synchronisation Utility Summary

GitHub: <https://github.com/lukecollins/file-delta-relay/tree/lightweight-portable>

1. Problem & Constraints

Design a client-side utility that monitors specified local directories and synchronises their contents to a remote server over a link that is both bandwidth-limited and unstable. Four requirements: (1) detect file changes; (2) minimise bytes on the wire; (3) survive connection drops without losing or repeating work; (4) let the server verify it holds an exact copy. *This branch adds a deployment constraint: the target host may not permit installing compiled extension wheels, so the implementation is pure Python standard library plus requests (client) and flask (server).*

2. Architecture Summary

One structure pays three requirements at once: a content-addressed chunk store with per-file manifests. Files are split into fixed 256 KiB chunks, each chunk is named by the BLAKE2b-256 digest of its own bytes, and a file's manifest is the ordered list of those names. Deduplication then becomes set membership rather than diffing (bandwidth); re-uploading a chunk is harmless, so resume after a drop is the same set-difference query asked again, not a protocol feature with its own state machine (reliability); and defining a file's identity as the hash of its ordered chunk hashes lets the server prove end-to-end equality without re-reading assembled bytes (integrity). Change detection, the fourth requirement, is a local SQLite state database that is checked against file metadata, with the content hash as the final arbiter when metadata is inconclusive. The client drives the protocol; the server is a small *verify-on-write* store; it recomputes every chunk digest on receipt and validates every *commit*, trusting no client-side state.

```
[scan/check] > [classify()] > [chunk_manifest()] > [get_missing_chunks] > [upload missing, AIMD window] > [commit_file]
```

3. Core Requirements Mapping

3A. Change Detection. A reconciliation pass *stats* each file and compares (*st_size*, *st_mtime_ns*) against the state DB, keyed by path. *classify()* is a pure decision kernel returning *NEW*, *STAT_CHANGED*, *INTERRUPTED* (a previous sync never completed), *BOUNDARY*, or *None* (provably unchanged). If a file's *st_mtime_ns* falls within a 2s guard window of the last content verification, it is *BOUNDARY*: inconclusive, so it is re-verified using the content hash rather than skipped. The policy errs toward false positives, which are nearly free since deduplication turns them into one local read and zero new bytes on the wire, and never toward false negatives. **Evidence:** *test_change_detection.py* (branch coverage of the kernel, including the exact guard-window boundary) and *test_classify_properties.py* (Hypothesis property tests: the *BOUNDARY/None* cutoff holds for all inputs, the verdict is clock-independent, and *classify()* never raises).

3B. Bandwidth. Fixed-size chunking, content-addressed deduplication, a batched pre-upload *get_missing_chunks()* query, and zlib (level 1) compression ensure only novel chunks are uploaded. **Evidence:** *02_bandwidth_efficiency.png* (data in *local_metrics.json*): a 50 KB edit to a 3 MB file uploads exactly one 256 KiB chunk (262,230 bytes, including compression), a ~92% reduction compared to re-sending the file.

3C. Reliability. Chunk uploads are idempotent and content-addressed, so retries are harmless. The *manifest* and its chunked state persist in the DB, so a crashed or disconnected sync is picked up as *INTERRUPTED* and resumes by re-asking the same set-difference question, with no bespoke resume protocol. Upload concurrency is governed by an *AIMD* controller (the window grows on healthy round trips, halves on loss or latency spikes), and the HTTP transport fails over across multiple servers using health probes. **Evidence:** *03_resume_recovery.png*: an interrupted 6 MB sync sent 3 chunks; the resumed pass sent exactly the remaining 21 of 24 bytes, byte-exact, with nothing sent twice. *02_interrupted_resume.log*: convergence through 30% packet loss and a mid-sync blackout, with no chunk ever stored twice (idempotent *puts*). *04_failover_sequence.png*: syncs continue on the secondary while the primary is down, then fail back to the secondary.

3D. Integrity. The server recomputes each chunk's BLAKE2b digest on receipt and rejects mismatches (HTTP 409) before writing (*verify-on-write*). At *commit*, which is atomic under a single lock, it recomputes the *file_hash* from the ordered chunk-hash list, rejects a non-matching *file_hash*, and verifies every referenced chunk still exists, returning the *missing* set for the client to re-upload and retry. On the client, a *torn-read guard* stats the file before and after chunking and abandons the *manifest* if the file changes underneath it, so any torn read the stat detects is abandoned rather than committed. An edit preserving both *st_size* and *st_mtime_ns* in the same timestamp tick is invisible to any stat-based check; the guard window catches it by re-verification on the next pass. **Evidence:** *test_integration_http.py* asserts byte-exact equality of the server's reassembled copy after initial sync, edits, rename, resume, and failover, and that a restarted client reuses its persisted state and re-syncs nothing (11 end-to-end checks).

4. Code (Verbatim)

4A. *classify()* decision kernel (*change_detection.py*; partially elided):

```
def classify(rec: "FileRec | None", st: _StatLike, now_ns: int) -> Reason | None:
    if rec is None:
        return Reason.NEW
    if rec.state != "synced":
        return Reason.INTERRUPTED
    if (st.st_size, st.st_mtime_ns) != rec.key:
        return Reason.STAT_CHANGED
    if st.st_mtime_ns >= rec.verified_at_ns - GUARD_NS:
        return Reason.BOUNDARY # same-tick write could hide here. The
        # anchor is when content was last READ and matched, never commit
        # time: a slow upload must not widen the blind spot between
        # chunking and commit.
    return None
```

Why chosen: Pure function of (db record, stat, clock); guard arithmetic is unit- and property-testable without a filesystem. Returns *None* only if stat proves the file unchanged, avoiding unnecessary content reads. Uncertain *st_mtime_ns* triggers content-hash re-verification to prevent false negatives.

4B. `_sync_file()` Core transfer loop (*single file transfer.py*, `_sync_file`; error handling elided):

```
def _sync_file(db: "StateDB", srv: SyncServer, path: str,
              controller: "AIMDController | None" = None) -> None:
    known = db.prev_hashes(path)
    ask = [h for _, _, h in manifest if h not in known]

    try:
        for _ in range(MAX_COMMIT_ROUNDS):
            missing = set(srv.get_missing_chunks(ask))  # one batched round trip
            _upload(srv, path, [c for c in manifest if c[2] in missing], controller)
            result = srv.commit_file(
                path=path,
                file_hash=file_hash,
                chunk_hashes=[h for _, _, h in manifest],
                size=st.st_size, mtime_ns=st.st_mtime_ns, mode=st.st_mode,
            )
            if result.ok:
                break
        ask = result.missing  # lost a race with server GC
```

Why chosen: Protocol runs in a single loop: local pre-diff with the last synced *manifest*, one batched *get_missing_chunks()* round trip, upload of only the gap under the *AIMD* window, and a self-healing *commit*. If validation finds missing references, the server reports them and the loop re-uploads and retries.

5. Results

Metric	main branch	lightweight-portable (this branch)
Dependencies	3 C/Rust extensions (fastcdc, blake3, zstd)	Pure Python (zero compiled wheels)
Chunking strategy	Content-defined (~256 KiB avg)	Fixed-size (256 KiB)
50 KiB Insert (8 MiB file)	1 chunk resent (3.0%)	33 of 33 chunks (100%)
50 KiB Overwrite	3 chunks resent (8.8%)	1 chunk resent (3.1%)
50 KiB Append	1 chunk resent (3.0%)	1 chunk resent (3.0%)
Peak Client CPU	56%	87%
Degraded Link (30% loss)	Timed out	Converged

Bandwidth savings reach ~92% for minor edits while guaranteeing exact, chunk-granular recovery without data duplication during connection drops. Efficiency relies on fixed-size chunking; this strategy excels at overwrites and appends but incurs overhead on data inserts due to cascading boundary shifts.

6. Scope & Trade-offs

Implementation targets locked-down or air-gapped hosts where *pip install* of native wheels (*fastcdc*, *blake3*, *zstd*) is restricted due to missing C/Rust toolchains, security policies, or lack of PyPI access. The pure-Python stack ensures the chunking, hashing, and compression pipeline remains inspectable and dependency-free, facilitating audits in restricted environments.

Known limitations: fixed-size chunking (256 KiB) trades insert-shift efficiency for portability. A single byte inserted near the front of a file cascades every subsequent boundary, resending 100% of the file, whereas content-defined chunking (CDC) would resend only ~3%. However, overwrites and appends (far more frequent in practice) see the same ~92% bandwidth saving as CDC. CPU utilisation also peaks higher (87% vs. 56%) because *stdlib's zlib* and *blake2b* are slower per byte than *zstandard* + *blake3*; this is an acceptable cost for zero compiled dependencies.

Operational extensions: beyond the core sync, the client supports *SYNC_DELETE_AFTER* (which removes the source file immediately after the server confirms a successful commit, leaving no local trace) and a configurable state-DB path (*SYNC_STATE_DB*) that can be relocated to a benign application directory. Server-side tombstones do not trigger garbage collection of the underlying chunks, ensuring that deleted files remain recoverable from the server's deduplicated pool: a deliberate retention choice suitable for collection or exfiltration workflows.

7. Reproducibility

The repository includes a self-contained demo harness (*sync-demo*) that bootstraps its own virtualenv and starts both a real *Flask* server and a client instance – no Docker required for the quick suite.

```
cd sync-demo
./demo.sh quick      # no Docker: real server + client; 14 unit,
# 11 integration, 4 auth checks in a few seconds
./demo.sh full       # Docker starts with network-fault injection:
# scenarios 1-3 plus a written report
```

A fresh clone requires only Python 3.12+, OpenSSL, and network access for the one-time certificate and virtualenv bootstrap. The entire evidence pipeline is regenerated step by step via *run_full_evidence.sh*; *make_plots.py* produces all figures, and *publish.sh* snapshots the outputs per branch. Full instructions are in *the evidence/README.md file*.